

System Programming Project 3

담당 교수 : 김영재

이름 : 신지원

학번 : 20211547

1. 개발 목표

동시성을 가진 주식 서버를 개발하는 것이 가장 주된 목표다. 하나의 서버가 클라이언트를 처리해야 하기 때문에, 동기적으로 프로그래밍을 구현할 경우엔 클라이언트의 UX와 서버 리소스가 저하되게 된다. 따라서 동시성을 고려한 서버 프로그래밍의 필요성을 고려하여 개발하도록 하였다.

첫 번째는 Event-driven 방식으로 구현하고, 두 번째는 Thread-Based 방법으로 구현할 것이다. 또한 이 둘의 성능 차이를 비교하여 차이를 알아보도록 하였다..

2. 개발 범위 및 내용

A. 개발 범위

1. Task 1: Event-driven Approach

select() 함수와 fd_set 을 활용하여 이벤트 기반의 동시성 처리 서버를 구현한다. 이 방식에서는 서버가 단일 스레드에서 순차적으로 이벤트를 감지하고 처리하므로, 프로세스 간의 충돌이나 자원 경합에 대해 상대적으로 안전하다. 단, select() 방식은 감시할 수 있는 파일 디스크립터의 개수(FD_SETSIZE)에 제한이 있으며, 이는 운영체제 설정에 따라 달라진다. 따라서 실사용 시 동시 접속자의 수를 고려한 설정 조정이 필요하다.

2. Task 2: Thread-based Approach

스레드를 활용하여 동시성을 지원하는 서버를 구현하였다. 이 방식은 여러 개의 스레드가 동시에 실행되며, 공유 자원에 접근할 수 있기 때문에 세마포어를 이용한 동기화가 필수적이다. 본 프로젝트에서는 주식의 수량 정보와 연결 요청 큐가 대표적인 공유 자원이며, 이들에 대해 동시 접근을 제어하여 데이터의 무결성을 보장하였다.

서버는 Master-Worker 구조를 따르며, main 스레드가 클라이언트로부터 들어오는 connection 요청을 수락하고, 이를 버퍼에 저장하면, 나머지 Worker 스레드들이 해당 요청을 꺼내 처리하는 방식으로 동작한다. 이를 위해 sbuf 라는 구조를 사용하였고, 이 구조에는 세 가지 종류의 세마포어가 함께 사용된다. mutex 는 스레드 간의 상호 배제를 보장하며, slot 은 버퍼에 남은 공간을 나타내고, item 은 버퍼에 저장된 유효한 요청의 수를 의미한다.

또한 주식 정보에 대해서도 별도의 mutex 세마포어를 사용하여 여러 스레드가 동시에 접근하지 못하도록 하였다.

전체 흐름은 다음과 같다. main 스레드는 클라이언트의 연결을 수락한 후 그 connection file descriptor 를 sbuf 에 추가한다. 만약 sbuf 가 가득 찼다면 main 스레드는 slot 세마포어에 의해 일시적으로 대기 상태가 된다. 반면 Worker 스레드는 item 세마포어가 0 이 될 때까지 대기하며, 유효한 항목이 존재할 경우 sbuf 에서 fd 를 꺼내 실제 클라이언트 요청을 처리한다. 이와 같이 main 과 Worker 스레드는 각각 blocking 과 unblocking 을 반복하면서, 서로 간섭하지 않고 안정적으로 병렬 처리를 수행하게 된다.

이 구조를 통해 서버는 많은 수의 클라이언트 요청을 동시에 효율적으로 처리할 수 있으며, 각 스레드는 race condition 없이 안정적으로 동작할 수 있다. sbuf 와 세마포어를 기반으로 한 이러한 Master-Worker 모델은 시스템 자원을 효율적으로 활용하고, 성능 저하 없이 확장 가능한 동시성 처리를 가능하게 한다.

3. Task 3: Performance Evaluation

두 가지 방식(Task 1: 이벤트 기반, Task 2: 스레드 기반)으로 동시성 서버를 구현한 뒤, 각 방식의 성능 차이를 비교하고 설정 값을 다르게 적용했을 때의 성능 변화를 함께 분석하고자 한다.

Task 1 (Event-driven Approach)의 경우, 논리적으로 동시성을 지원하지만 실제로는 하나의 프로세스가 모든 요청을 순차적으로 처리하므로 설정 측면에서 조정 가능한 요소가 많지 않다. 따라서 테스트에서는 multiclient 프로그램을 활용하여 클라이언트 수를 다르게 구성하고, 요청 유형을 다양하게 조합하여 서버가 동시 요청을 얼마나 안정적으로 처리하는지를 확인할 것이다. 특히 클라이언트 수가 증가할수록 처리율이 어떻게 변하는지를 관찰하여 이벤트 기반 모델의 한계를 측정한다.

반면, Task 2 (Thread-based Approach)에서는 설정에 따라 성능 차이가 뚜렷하게 발생할 수 있다. 가장 대표적인 예는 Worker Thread 의 수이다. 예를 들어 워커 스레드가 10 개일 때와 100 개일 때 각각 1000 명의 클라이언트 요청을 처리한다면, 전자의 경우 각 스레드가 평균 100 명씩,

후자의 경우 10 명씩 담당하게 된다. 만약 워커 스레드 수가 클라이언트 수보다 과도하게 많을 경우에는 유휴 스레드가 생기게 되며, 이는 오히려 자원 낭비로 이어질 수 있다. 따라서 워커 스레드 수를 조절하면서, 처리량, 응답 시간, CPU 사용률 등을 비교하여 가장 효율적인 구성 값을 찾는다.

B. 개발 내용

- Task1 (Event-driven Approach with select())

- ✓ 기반 방식으로 서버를 구현할 때, 클라이언트 연결은 fd_set을 통해 관리하게 된다. 이때 사용하는 주요 데이터 구조는 read_set과 ready_set이며, 각각 클라이언트로부터 읽을 준비가 된 소켓을 감지하고, 실제로 처리 가능한 요청이 감지된 소켓을 식별하는 데 사용된다. 서버는 클라이언트 연결 정보를 clientfd 배열에 저장하며, 이를 통해 연결된 클라이언트 소켓들을 효율적으로 추적한다.

이벤트가 감지되면 read_set에서 복사한 ready_set을 통해 어떤 파일 디스크립터가 요청을 보냈는지 확인하고(FD_ISSET(clientfd[i], &ready_set)), 요청이 들어온 경우 해당 클라이언트의 처리를 수행하게 된다. 이 방식은 하나의 프로세스가 여러 클라이언트의 요청을 순차적으로 처리하지만, I/O 이벤트에 반응하여 다중 클라이언트를 지원할 수 있다는 점에서 효율적이다.

epoll은 이러한 select() 방식의 한계를 개선하기 위해 고안된 고성능 이벤트 감지 방식이다. select()는 내부적으로 모든 파일 디스크립터를 순회하며 이벤트를 검사해야 하기 때문에 연결 수가 많아질수록 성능이 급격히 저하되는 문제가 있다. 반면 epoll은 커널 수준에서 이벤트를 관리하므로, 사용자 레벨 코드에서 디스크립터를 매번 순회할 필요가 없고, 더욱 효율적인 이벤트 분배가 가능하다.

다만 epoll은 초기 설정과 사용법이 상대적으로 복잡하다는 단점도 있다. 반면 select()는 구조가 단순하고 빠르게 구현할 수 있기 때문에, 소규모 서버나 프로토타이핑 단계에서 여전히 유용하게 활용될 수 있다. 결국 시스템의 규모와 목적, 유지보수의 난이도 등을 고려하여 상황에 맞는 방식을 선택하는 것이 중요하다.

- Task2 (Thread-based Approach with pthread)

Thread 기반 방식에서는 Master-Worker 구조를 채택하여 서버를 구현하였다. 이 구조에서 Master 스레드는 클라이언트로부터의 연결 요청을 수락(Accept)하고, 해당 연결 정보를 공유 버퍼인 sbuf에 저장한다. Worker 스레드는 이 sbuf를 통해 connection 정보를 전달받고 클라이언트의 요청을 처리하게 된다.

Master 스레드는 클라이언트 요청을 수락한 뒤, 연결된 소켓 디스크립터(connfd)를 sbuf에 삽입한다. 만약 sbuf에 여유 공간이 없을 경우(slot 세마포어가 0이 되면), Master 스레드는 대기 상태에 들어가고, 공간이 생기면 다시 작업을 이어간다. 이 과정은 sbuf_insert() 함수를 통해 구현되며, mutex, slots, items 세 가지 세마포어를 이용하여 스레드 간의 동기화를 보장한다.

Worker 스레드는 서버 시작과 동시에 지정된 수만큼 생성되며, 각 스레드는 detach 상태로 실행되어 종료 시 자원을 자동으로 반환한다. 이 스레드들은 무한 루프를 돌며 sbuf에서 처리할 connection이 있는지를 검사하고, 요청이 들어오면 처리에 들어간다. Worker 스레드는 별도로 관리되지 않지만, sbuf를 중심으로 item, slot, mutex 세마포어를 통해 접근 순서와 자원 경합을 제어함으로써 race condition을 방지할 수 있다.

이러한 구조는 요청 수가 많을수록 스레드가 병렬로 작업을 분담하므로 처리량을 높이는 데 유리하며, select() 기반보다 더 나은 응답성과 확장성을 기대할 수 있다. 단, 스레드 수가 너무 많거나 공유 자원에 대한 동기화가 제대로 이루어지지 않으면 오히려 병목이 발생할 수 있으므로 적절한 스레드 수와 자원 관리가 핵심이다.

- Task3 (Performance Evaluation)

- ✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술
- ✓ Configuration 변화에 따른 예상 결과 서술

C. 개발 방법

- Task1 (Event-driven Approach with select())

Task 1에서는 select() 기반의 동기적 I/O 멀티플렉싱 기법을 활용하여, 하나의 서버 프로세스가 다수의 클라이언트 요청을 순차적으로 처리할 수 있도록 구현하였다. 서버는 초기화 시점에 stock.txt로부터 주식 정보를 읽어와 메모리에 이진 탐색 트리(Binary Search Tree) 형태로 적재하고, 클라이언트 요청이 들어올 때마다 해당 정보를 기반으로 명령을 처리하는 구조이다. 이때 주식에 대한 구조체와, 클라이언트에 대한 구조체로 각각 stock, Pool 을 선언하였다. 끝은 아래와 같다.

```
typedef struct {
    int id;
    int left_stock;
    int price;
} Stock;

typedef struct {
    int maxfd;
    fd_set read_set;
    fd_set ready_set;
    int nready;
    int clientfd[FD_SETSIZE];
    rio_t clientrio[FD_SETSIZE];
} Pool;
```

서버는 select() 시스템 콜을 사용하여 연결 요청과 클라이언트 명령을 감지한다. 새로운 연결이 발생하면 Accept()로 연결을 수락하고, 해당 소켓 디스크립터를 fd_set에 등록한다. 클라이언트로부터 명령이 수신되면 Rio_readlineb() 함수로 입력을 읽고, 명령어 종류를 판별하여 buy, sell, show, exit 요청을 처리한 뒤, 결과 메시지를 Rio_writen()으로 응답하는 방식이다.

주요 모듈로는 클라이언트 연결을 관리하는 pool 구조체가 있으며, 여기에 각 클라이언트의 소켓 디스크립터 배열, 버퍼(rio_t), fd_set(read/ready set) 정보가 포함된다. 주식 정보는 insert_stock, search, show_stock, save_stock, load_stock 함수들을 이용해 이진 탐색 트리로 관리된다.

Task 1에서 서버는 select()를 통해 이벤트가 발생한 소켓을 순차적으로 처리하므로 동시 접근 문제는 발생하지 않으며, Readers-Writers 문제도 고려하지 않았다.

추가적으로, buy, show, exit 명령어의 응답 결과에서 첫 줄이 클라이언트 측에서 간혹 누락되는 현상이 있었다. 이를 해결하기 위해 모든 응답 메시지 끝에 명시적으로 "endWn"을 추가하여 출력의 경계를 명확히 설정하였다. 하지만 이로 인해 출력 양식이 멀티클라이언트 테스트용 예시 형식과 다르게 되어, 과제에서 제공된 multiclient 프로그램으로는 테스트를 진행하지 못했다.

- Task2 (Thread-based Approach with pthread)

Task 2에서는 Thread 기반의 동시성 처리 모델을 구현하기 위해 pthread를 활용하였다. Task 1에서 사용한 select() 기반의 I/O 멀티플렉싱 구조는 하나의 프로세스가 순차적으로 클라이언트 요청을 처리하는 방식이었으나, Task 2에서는 멀티스레드 환경을 도입함으로써 다수의 클라이언트 요청을 병렬적으로 처리할 수 있도록 개선하였다.

특히 주식 정보를 이진 탐색 트리 (Binary Search Tree, BST) 형태로 저장하고 관리하였다. 각 주식 항목은 id, left_stock, price를 포함하는 구조체로 정의되고, 노드 단위로 BST에 삽입되거나 탐색된다. 관련 구조체 정의는 아래와 같다

```
struct stock {
    int id;
    int left_stock;
    int price;
};

typedef struct node{
    struct stock data;
    struct node *left;
    struct node *right;
} node;

typedef struct BST{
    node *root;
} BST;
```

또한, 클라이언트 요청 큐를 위한 버퍼는 다음과 같은 구조체 sbuf_t로 정의하였다: 해당 구조체는 연결된 클라이언트의 소켓 디스크립터를 큐 방식으로 저장하며, 서버의 main thread는 클라이언트가 접속할 때마다 sbuf_insert() 함수를 통해 이 큐에 소켓을 넣고, worker thread들은 sbuf_remove()를 통해 이를 꺼내 명령을 처리한다. 구조는 아래와 같다.

```
typedef struct {
    int *buf;
    int n;
    int front;
    int rear;
    sem_t mutex;
    sem_t slots;
    sem_t items;
} sbuf_t;
```

서버는 시작 시점에 stock.txt로부터 주식 정보를 읽어와 이진 탐색 트리(Binary Search Tree, BST)에 적재한다. BST는 각 주식을 노드로 가지며, id 값을 기준으로 정렬되어 있어 빠른 탐색이 가능하다. 이후 서버는 일정 수의 스레드를 생성하고, 각 스레드는 클라이언트 연결을 처리하기 위한 thread pool의 worker로 동작한다. 클라이언트가 연결되면 main thread가 연결된 소켓 디스크립터를 요청 버퍼(sbuf)

에 삽입하고, 대기 중이던 worker thread가 이를 꺼내서 실제 요청을 처리하는 방식이다.

요청 처리를 담당하는 work() 함수는 클라이언트로부터 명령어를 한 줄 읽고, 명령어 종류에 따라 buy, sell, show, exit 등을 수행한다. 이때 BST는 모든 스레드가 공유하는 구조이기 때문에, 데이터 무결성을 보장하기 위해 mutex 세마포어를 활용하였다.

스레드는 detach 모드로 실행되며, 메모리 누수가 발생하지 않도록 처리하였다. 또한 서버 전체 종료 시에는 SIGINT 시그널 핸들러를 등록하여 서버가 강제 종료되더라도 stock.txt에 현재 주식 정보가 저장되도록 하였다. 이 과정에서는 요청 버퍼 메모리를 해제하고, BST 구조체도 순회하며 동적 메모리를 반환한 후 종료 메시지를 출력하고 프로세스를 종료한다.

Task 2에서는 select()를 사용하지 않고도 다수의 클라이언트 요청을 효율적으로 처리할 수 있도록 설계되었으며, 실제 테스트에서 여러 클라이언트가 동시에 buy 나 sell 명령을 보내더라도 데이터의 일관성이 유지되는 것을 확인하였다.

이외 관련 함수들은 코드에 주석을 첨부하였다.

3. 구현 결과

- Task1 (Event-driven Approach with select())

1) 단일 클라이언트

```

[cse20211547@cspro9:~/project3/task_1$ ./stockclient 172.30.10.11 60055
[ show
1 99885 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100023 813
9 100151 531
10 99863 475
[buy 1 1
[buy] success
[show
1 99884 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100023 813
9 100151 531
10 99863 475
[sell 1 1
[sell] success
[show
1 99885 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100023 813
9 100151 531
10 99863 475

```

2) 멀티 클라이언트

-

- Task2 (Thread-based Approach with pthread)

1) 단일 클라이언트

```
[cse20211547@cspro9:~/project3/task_2$ ./stockclient 172.30.10.11 60055
[ show
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100014 813
9 100155 531
10 99869 475
[buy 1 1
[buy] success
[show
1 99882 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100014 813
9 100155 531
10 99869 475
[sell 1 1
[sell] success
[show
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100014 813
9 100155 531
10 99869 475
[exit
```

2) 멀티 클라이언트 (3명으로 선언)

```

[cse20211547@cspro9:~/project3/task_2$ ./multiclient 172.30.10.11 60055 3
child 2954824
child 2954825
child 2954826
[buy] success
[buy] success
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100004 813
9 100151 531
10 99869 475
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100004 813
9 100151 531
10 99869 475
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100004 813
9 100151 531
10 99869 475
1 99883 638
2 100427 423
3 99250 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100004 813
9 100151 531
10 99869 475
[sell] success
1 99883 638
2 100427 423
3 99254 591
4 101207 200
5 99997 613
6 100132 588
7 100708 791
8 100004 813

```

4. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

4-1. buy, sell, show 의 명령어만 입력된 경우 (task_1 의 multiclient.c 파일을 수정하여 성능평가 진행함)

배경)

- 본 실험은 buy, sell, show 명령만을 사용한 요청 처리 성능을 측정하기 위한 것으로, 클라이언트 수는 100명, 각 클라이언트당 요청 수는 10,000건으로 고정하였다. Task 2는 pthread 기반의 멀티스레드 방식으로 구현되어 있으며, 이를 위해 worker thread의 수는 클라이언트 수와 동일하게 100개로 설정하였다.

예측)

- Task 2는 공유 자원에 대한 동시 접근을 제어하기 위해 mutex를 사용하므로, 동기화 오버헤드가 발생할 수 있다. 이에 따라 Task 1보다 처리 시간이 더 길어질 것으로 예상하였다. 특히, buy와 sell 명령은 주식 데이터에 대한 변경 연산이 포함되어 있어 mutex의 영향을 받을 가능성이 크다. 반면 show 명령은 읽기 전용이므로 상대적으로 빠를 것으로 예상되었다.

결과)

	buy			sell			show		
	#1	#2	#3	#1	#2	#3	#1	#2	#3
task_1	71	70	72	69	70	71	72	70	71
task_2	70	70	71	70	71	70	70	70	70

결과 분석)

- 예상과 달리 Task 2에서의 mutex 사용이 성능에 큰 영향을 주지 않았으며, Task 1과 Task 2 모두 높은 처리 효율을 보였으며, 두 방식 간 유의미한 성능 차이는 나타나지 않았다.

4-2. clinet 가 증가될 때 (TASK_1)

배경)

- Task 1에서는 event-driven 방식의 서버에서 클라이언트 수를 100명부터 1000명까지 점진적으로 증가시키며 처리 성능의 변화를 관찰하였다. 각 클라이언트는 동일하게 10,000개의 요청을 전송하고, 서버는 이를 모두 처리한 후 종료된다.

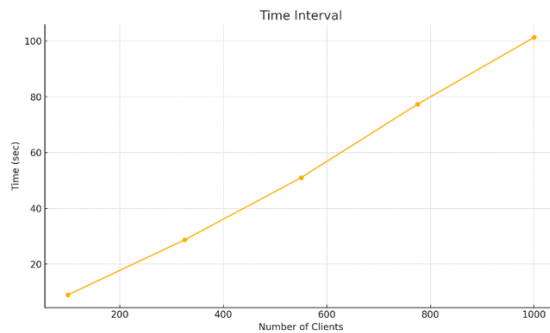
예측)

- 클라이언트 수가 증가하면 select() 기반의 I/O 멀티플렉싱에서 감시해야 할 파일 디스크립터 수가 많아지고, 순회 시간 및 fd_set 관리에 따른 오버헤드도 증가할 것으로 예상된다. 이에 따라 전체 처리 시간이 점진적으로 증가하고, 단위 시간당 처리율(throughput)은 감소할 것으로 예측된다.

결과)

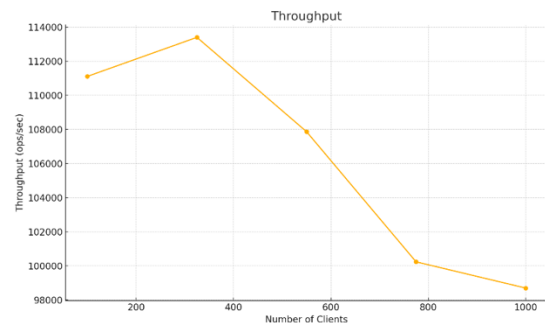
✓ TimeInterval

#Client	#1	#2	#3	avg
100	9	9	9	9.000
325	29	29	28	28.667
550	51	52	50	51.000
775	78	76	78	77.333
1000	102	100	102	101.333



✓ Throughput

#Client	#1	#2	#3	avg
100	111111.1	111111.1	111111.1	111111.1
325	112069	112069	116071.4	113403.1
550	107843.1	105769.2	110000	107870.8
775	99358.97	101973.7	99358.97	100230.5
1000	98039.22	100000	98039.22	98692.81



결과분석)

- 실험 결과, 예측한 바와 같이 클라이언트 수가 증가함에 따라 전체 처리 시간 (Time Interval)은 선형적으로 증가하는 경향을 보였다. 특히 100명에서 1000명까지 증가할수록 평균 처리 시간이 9초에서 101.3초로 약 11배 이상 상승하였다.
- Throughput(초당 처리 요청 수)의 경우, 클라이언트 수가 적을 때 가장 높은 수치를 기록했으며(예: 111,111.1 ops/sec), 클라이언트 수가 증가할수록 점진적으로 감소하는 양상을 보였다. 이는 select() 호출 시 매번 fd를 순회하며 이벤트를 감지해야 하기 때문에, 감시 대상이 많아질수록 서버의 응답성이 저하되기 때문이다.

4-3. Worker Thread 와 Client 수의 관계 (TASK_2)

배경)

- Task 2에서는 Master-Worker 구조를 기반으로, 요청이 발생하면 이를 연결 큐에 저장하고, worker thread가 이를 순차적으로 꺼내 처리하는 방식으로 동작한다.

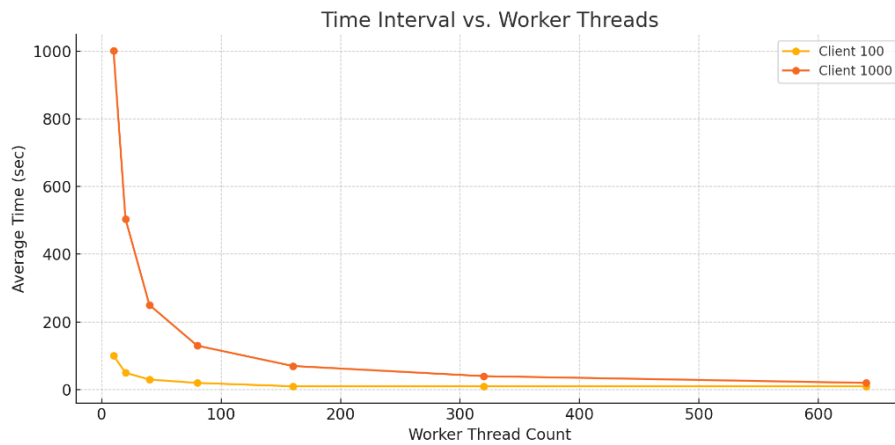
해당 실험에서는 Worker Thread의 수를 10에서 640까지 배수로 늘려가며, 동시에 클라이언트 수를 100명, 1000명으로 설정하여 평균 응답 시간(Time Interval)을 측정하였다. 각 클라이언트는 동일한 수의 요청을 보내며, 각 환경에서 3회의 실험을 통해 평균값을 산출하였다.

예측)

- Thread 수가 적으면 클라이언트 요청이 큐에 대기하면서 병목이 발생하므로 처리 시간이 길어질 것이다. 반대로 Thread 수가 충분하거나 많아지면 클라이언트 요청을 병렬로 빠르게 처리할 수 있어 처리 시간은 감소할 것이다.

결과)

#Worker Thread	#Client (100) avg	#Client (1000) avg
10	100.33	1000.67
20	50	504.33
40	30	250.33
80	20	130
160	10	70
320	10	40
640	10	20



결과분석)

- 클라이언트 수보다 적은 수의 Thread를 사용할 경우, 모든 요청이 순차적으로 처리되어 시간이 오래 걸리는 반면, Thread 수가 클라이언트 수 이상이 되면 병렬 처리에 가까워지면서 처리 시간이 고정값(요청 횟수 × 요청 간 간격)에 수렴하는 것을 확인할 수 있었다.
- 특히 클라이언트 1000명 실험에서는 Thread 수가 적을수록 처리 시간이 증가하며, Thread 수가 많아질수록 처리 시간이 급격히 줄어드는 양상을 보였다.

이 결과는 Task 2의 Master-Worker 구조에서 Thread 개수의 증가가 처리 성능에 결정적 영향을 준다는 점을 잘 보여준다.